

## Camp Overview

This is a full plan for a **Arduino Robotics & Electronics** camp of **55 children** on 6th August 2026 at Fireside Group. The intake is split into age bands — 7-9 yrs (20), 10-12 yrs (23), 13-15 yrs (12) — and each band into small groups so every child works at the right level and gets plenty of hands-on time. The sections below give the group split, staffing, the master timetable, and a tailored programme for each band.

## Groups at a Glance

Each group stays together all day with its own lead facilitator, so no child is ever in a group larger than the target size.

| AGE BAND  | GROUP               | SIZE        |
|-----------|---------------------|-------------|
| 7-9 yrs   | 7-9 yrs · Group A   | 10 children |
| 7-9 yrs   | 7-9 yrs · Group B   | 10 children |
| 10-12 yrs | 10-12 yrs · Group A | 8 children  |
| 10-12 yrs | 10-12 yrs · Group B | 8 children  |
| 10-12 yrs | 10-12 yrs · Group C | 7 children  |
| 13-15 yrs | 13-15 yrs · Group A | 6 children  |
| 13-15 yrs | 13-15 yrs · Group B | 6 children  |

## Staffing & Ratios

With **55 children** in **7 groups**, this plan needs **12 adults** on the day — an overall ratio of about **1 adult to 4.6 children** (target 1:10 or better).

| ROLE                              | PEOPLE    |
|-----------------------------------|-----------|
| Lead facilitators (one per group) | 7         |
| Floating helpers                  | 3         |
| Camp coordinator                  | 1         |
| First-aider                       | 1         |
| <b>Total adults</b>               | <b>12</b> |

## Master Day Timetable

The whole camp runs on one clock. Groups start and stop together so breaks, lunch and the showcase stay in sync.

| TIME          | ACTIVITY                                                                         |
|---------------|----------------------------------------------------------------------------------|
| 08:30 – 09:00 | Arrival, registration & name badges                                              |
| 09:00 – 09:20 | Opening assembly — welcome, safety brief, split into groups (all bands together) |
| 09:20 – 10:45 | <b>Project Block 1</b> — each group in its own space                             |
| 10:45 – 11:05 | Morning break & snack (staggered so the space doesn't crowd)                     |
| 11:05 – 12:30 | <b>Project Block 2</b> — build continues / next activity                         |
| 12:30 – 13:15 | Lunch & free play (supervised, outdoors if possible)                             |
| 13:15 – 14:30 | <b>Project Block 3</b> — finish builds, test & debug                             |
| 14:30 – 14:50 | Showcase prep — groups tidy kits and rehearse a 60-second demo                   |
| 14:50 – 15:20 | Showcase & demos — every group presents to the whole camp                        |
| 15:20 – 15:30 | Closing circle, certificates & departure                                         |

## Programme by Age Band

Each band works through age-appropriate TinkerWithMe projects. Every project below includes its materials, step-by-step instructions and full code/build steps so a facilitator can run it directly.

## 7-9 yrs — 20 children in 2 group(s)

**Projects for this band:** Traffic light simulation, Ultrasonic distance sensor, Basic alarm system. Each group's lead facilitator preps the kit, keeps the group to the master timetable, and watches for the common mistakes noted in each project.

### Traffic light simulation (*Beginner · ~60 min*)

#### Materials

---

- 1 x Arduino Uno board (Nerokas.co.ke)
- 1 x USB cable (Type A to Type B) for programming (Nerokas.co.ke)
- 1 x Breadboard, half-size or full-size (Pixel Electronics.co.ke)
- 1 x Red LED, 5mm (Nerokas.co.ke)
- 1 x Yellow (or amber) LED, 5mm (Nerokas.co.ke)
- 1 x Green LED, 5mm (Nerokas.co.ke)
- 3 x 220 ohm resistors (colour bands: red-red-brown) (Pixel Electronics.co.ke)
- 4 x Male-to-male jumper wires (Nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (not purchased locally)

#### How It Works

---

Real traffic lights follow a fixed pattern: red means stop, green means go, and yellow (or amber) warns drivers that the light is about to change. This project recreates that pattern using three LEDs (Light Emitting Diodes) — one red, one yellow, one green — controlled by an Arduino.

An Arduino is a small computer that can turn electricity on and off very precisely, using a program (called a "sketch") that we write and upload to it. Each LED is connected to a different pin on the Arduino. In our code, we tell the Arduino things like "turn on the red LED, wait 5 seconds, turn it off, turn on the green LED..." and so on, in a loop that repeats forever.

LEDs only allow electricity to flow in one direction and they need just the right amount of current — too much and they burn out instantly. That is why each LED is paired with a 220 ohm resistor, which acts like a narrow gate that limits how much electricity can flow through, protecting the LED from damage.

The core programming idea here is **sequencing with timing**: the Arduino executes instructions one after another, and the `delay()` function tells it to pause for a set number of milliseconds before moving to the next instruction. By arranging `digitalWrite()` (turn a pin HIGH or LOW) and `delay()` commands in the right order inside a loop, we recreate the real-world timing of a traffic light.

## Step-by-Step Instructions

---

1. Place the breadboard in front of you with the long power rails running along the top and bottom edges.
2. Insert the red LED into the breadboard. Note that LEDs have one longer leg (the anode, positive) and one shorter leg (the cathode, negative). Place it so its two legs are in different rows of the breadboard.
3. Repeat for the yellow and green LEDs, spacing them out so each has its own set of rows and does not touch its neighbours.
4. For each LED, connect a 220 ohm resistor from the LED's shorter leg (cathode) row to the breadboard's ground (-) rail.
5. Run one jumper wire from the ground (-) rail on the breadboard to the GND pin on the Arduino.
6. Run a jumper wire from the red LED's longer leg (anode) row to digital pin 13 on the Arduino.
7. Run a jumper wire from the yellow LED's longer leg (anode) row to digital pin 12 on the Arduino.
8. Run a jumper wire from the green LED's longer leg (anode) row to digital pin 11 on the Arduino.
9. Double-check that no bare wires are touching each other on the breadboard, and that each LED has its own resistor in its path.
10. Connect the Arduino to the computer using the USB cable.
11. Open the Arduino IDE, select the correct board (Arduino Uno) and port from the Tools menu.
12. Copy the sketch from the "Full Code" section into a new sketch window.
13. Click the Upload button (right-facing arrow icon) and wait for "Done uploading" to appear.

14. Observe: the red LED should light for several seconds, then turn off as green lights up, then green turns off as yellow blinks briefly, then it returns to red — repeating in a continuous loop.
15. If an LED does not light, check that it is inserted the correct way round (long leg towards the Arduino pin side) before assuming the code is wrong.
16. Once working, ask students to time the sequence with a stopwatch and compare it to real traffic lights they have seen.

## Full Code

```
// Traffic Light Simulation
// Three LEDs (red, yellow, green) mimic a real traffic light sequence

// Assign each LED colour to an Arduino digital pin
const int redPin = 13;    // Red LED connected to pin 13
const int yellowPin = 12; // Yellow LED connected to pin 12
const int greenPin = 11;  // Green LED connected to pin 11

// Timing settings (in milliseconds) - easy to change later
const int redDuration = 4000; // Red light stays on for 4 seconds
const int greenDuration = 4000; // Green light stays on for 4 seconds
const int yellowDuration = 1000; // Yellow light stays on for 1 second

void setup() {
  // Set all three LED pins as OUTPUT so the Arduino can send power to them
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
}

void loop() {
  // --- RED PHASE: Stop ---
  digitalWrite(redPin, HIGH); // Turn red LED on
  digitalWrite(yellowPin, LOW); // Make sure yellow is off
  digitalWrite(greenPin, LOW); // Make sure green is off
  delay(redDuration); // Wait while red stays lit

  // --- GREEN PHASE: Go ---
  digitalWrite(redPin, LOW); // Turn red off
  digitalWrite(greenPin, HIGH); // Turn green on
  delay(greenDuration); // Wait while green stays lit

  // --- YELLOW PHASE: Prepare to stop ---
  digitalWrite(greenPin, LOW); // Turn green off
  digitalWrite(yellowPin, HIGH); // Turn yellow on as a warning
  delay(yellowDuration); // Wait a short time on yellow
  digitalWrite(yellowPin, LOW); // Turn yellow off before looping back to red

  // The loop() function repeats automatically, so it goes back to RED PHASE
}
```

## Common Mistakes & Fixes

| SYMPTOM                                                                    | LIKELY CAUSE                                                                              | FIX                                                                                                                                 |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| An LED does not light up at all                                            | LED inserted backwards (legs reversed)                                                    | Remove the LED, flip it around so the longer leg (anode) connects towards the Arduino pin, and reinsert                             |
| LED glows very faintly or flickers                                         | Loose jumper wire or resistor not fully pushed into breadboard holes                      | Press all wires and resistor legs firmly into the breadboard rows                                                                   |
| All LEDs stay off after uploading                                          | Wrong board or port selected in Arduino IDE, or upload failed silently                    | Check Tools > Board and Tools > Port, then re-upload and wait for "Done uploading" message                                          |
| Two LEDs light up at the same time when they should not                    | <code>digitalWrite(..., LOW)</code> line missing or commented out for the previous colour | Check each phase in the code turns off the LED(s) that should no longer be lit                                                      |
| Sequence timing looks wrong (too fast or too slow)                         | <code>delay()</code> values not matched to the intended real-world timing                 | Adjust the <code>redDuration</code> , <code>greenDuration</code> and <code>yellowDuration</code> constants at the top of the sketch |
| LED burns out (goes dark permanently, sometimes with a small pop or smell) | Missing or wrong-value resistor allowed too much current through the LED                  | Always use the 220 ohm resistor in series with each LED before power is applied                                                     |

## Discussion Questions

- Why do you think traffic lights use red, yellow, and green specifically, rather than any other three colours?
- What would happen in our code if we forgot to turn off the red LED before turning on the green one?
- Why does the resistor need to be placed in the circuit, and what might happen to the LED without it?
- In the code, what is the purpose of the `delay()` function, and what do you think would happen if all the delay values were changed to 100 milliseconds?
- Real traffic lights sometimes have a flashing yellow warning mode at night. How could you modify this program to add that behaviour?

## Extension Challenges

---

- **Easy:** Change the timing so red lasts 6 seconds and green lasts 3 seconds, then re-upload and observe the difference.
- **Medium:** Add a push button that, when pressed, immediately switches the light to red (like a pedestrian crossing button), then resumes the normal sequence afterward.
- **Hard:** Build a second traffic light for a perpendicular road using three more LEDs, and write code so that when one light is green, the other is red, simulating a real road intersection.

## Ultrasonic distance sensor (*Intermediate* · ~75 min)

### Materials

---

- 1 x Arduino Uno (or Uno-compatible board) (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x HC-SR04 Ultrasonic Distance Sensor module (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x Breadboard (half-size or full-size) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x LED (5mm, any colour, red recommended for a "warning" light) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x 220Ω resistor (for the LED) (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x Buzzer (optional, for an audible warning extension) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 8 x Male-to-male jumper wires (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x USB cable (A to B, for Arduino Uno) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x Laptop or computer with Arduino IDE installed (school-provided)
- Optional: ruler or tape measure for calibration activity (school-provided)

### How It Works

---

The HC-SR04 sensor works a lot like a bat finding its way in the dark, or a ship's sonar finding rocks underwater. It has two round "eyes" on the front — but they aren't eyes at all, they are a tiny speaker and a tiny microphone.

The speaker (called the **Trigger**) sends out a very short, very high-pitched sound — so high that humans cannot hear it. This sound travels through the air until it

hits an object (a hand, a wall, a book) and bounces back, just like an echo in a canyon. The microphone (called the **Echo**) listens for that bounced-back sound.

The Arduino keeps track of exactly how long it took for the sound to leave and come back — this is measured in tiny fractions of a second called microseconds. Since we know how fast sound travels through air (about 343 metres per second), we can do some simple maths to turn "time taken" into "distance to the object." The Arduino does this maths for us every time it takes a reading.

Once the Arduino knows the distance, it makes a decision: if the object is closer than a distance we choose (say, 10cm), it turns on an LED to warn that something is too close — just like a reversing sensor in a car. This teaches an important programming idea: using **sensor data to make decisions** with an `if` statement.

## Step-by-Step Instructions

---

1. Introduce the HC-SR04 module to students. Point out its four pins: VCC, Trig, Echo, and GND. Explain that Trig "shouts" and Echo "listens."
2. Place the HC-SR04 sensor on the breadboard (or hold it steady with wires — it does not need the breadboard if pins are used directly, but breadboard placement keeps wiring tidy).
3. Wire the sensor: VCC to Arduino 5V, GND to Arduino GND, Trig to Arduino digital pin 9, Echo to Arduino digital pin 10.
4. Insert the LED into the breadboard. Connect the LED's longer leg (anode) through the 220Ω resistor to Arduino digital pin 7. Connect the shorter leg (cathode) to a breadboard ground rail, then to Arduino GND.
5. Double-check all wiring before connecting power — a wrong connection on the HC-SR04 will not usually damage it, but good habits matter.
6. Connect the Arduino to the computer via USB. Open the Arduino IDE and paste in the sketch from the Full Code section below.
7. Select the correct board (Arduino Uno) and correct COM port under the Tools menu.
8. Upload the code. Open the Serial Monitor (set to 9600 baud) to see live distance readings printed in centimetres.
9. Ask students to slowly move a hand towards the sensor while watching the Serial Monitor numbers decrease.



10. Have students note the distance (in cm) at which the LED turns on. Ask them to guess this number before checking.
11. Challenge students to change the threshold value in the code (e.g. from 10cm to 20cm) and re-upload to see the warning distance change.
12. Wrap up by asking each group to explain, in their own words, how the sensor "sees" without eyes.

## Full Code

```
// Ultrasonic Distance Sensor with LED Proximity Warning
// HC-SR04 measures distance; LED lights up when an object is too close

const int trigPin = 9;      // Pin connected to HC-SR04 Trig (sends the sound pulse)
const int echoPin = 10;     // Pin connected to HC-SR04 Echo (receives the bounced sound)
const int ledPin = 7;       // Pin connected to the warning LED

const int warningDistance = 10; // Distance in cm below which the LED turns on

void setup() {
  pinMode(trigPin, OUTPUT); // Trig sends a signal out, so it is an OUTPUT
  pinMode(echoPin, INPUT);  // Echo receives a signal, so it is an INPUT
  pinMode(ledPin, OUTPUT);  // LED is controlled by the Arduino, so it is an OUTPUT
  Serial.begin(9600);       // Start Serial communication so we can see readings on the computer
}

void loop() {
  // Step 1: Send a short pulse out of the Trig pin to trigger a measurement
  digitalWrite(trigPin, LOW); // Make sure Trig starts LOW (off)
  delayMicroseconds(2);       // Tiny pause to ensure a clean signal
  digitalWrite(trigPin, HIGH); // Send the ultrasonic "shout"
  delayMicroseconds(10);      // Pulse must be at least 10 microseconds long
  digitalWrite(trigPin, LOW); // Turn the trigger off again

  // Step 2: Measure how long the Echo pin stays HIGH
  // pulseIn() waits for the pin to go HIGH, then times how long it stays HIGH (in microseconds)
  long duration = pulseIn(echoPin, HIGH);

  // Step 3: Convert the time into a distance
  // Sound travels about 0.0343 cm per microsecond
  // We divide by 2 because the sound travels to the object AND back
  float distanceCm = (duration * 0.0343) / 2;

  // Step 4: Print the distance so we can see it on the Serial Monitor
  Serial.print("Distance: ");
  Serial.print(distanceCm);
  Serial.println(" cm");

  // Step 5: Decide whether to warn with the LED
  if (distanceCm > 0 && distanceCm < warningDistance) {
    // distanceCm > 0 filters out invalid/zero readings from sensor errors
    digitalWrite(ledPin, HIGH); // Object too close - turn LED on
  } else {
    digitalWrite(ledPin, LOW);  // Object far away - turn LED off
  }
}
```

```

delay(200); // Small pause between readings so numbers don't scroll too fast to read
}

```

## Common Mistakes & Fixes

| SYMPTOM                                                                     | LIKELY CAUSE                                                                                    | FIX                                                                                                                                  |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Serial Monitor shows "0.00 cm" constantly, or numbers that never change     | Trig and Echo pins swapped, or wires loose in breadboard                                        | Double-check wiring: Trig to pin 9, Echo to pin 10. Push wires firmly into breadboard holes.                                         |
| LED never lights up even when an object is very close                       | LED inserted backwards (wrong polarity), or warningDistance value not matched to test distance  | Flip the LED so the longer leg (anode) faces the resistor/pin side. Try moving object closer than the warningDistance value in code. |
| Readings jump wildly or show huge/impossible numbers (e.g. 400+ cm indoors) | Sensor pointed at a soft, angled, or sound-absorbing surface (like fabric), or nothing in range | Point sensor at a flat, hard surface directly in front of it, within 2cm-400cm range.                                                |
| Nothing appears in Serial Monitor at all                                    | Wrong baud rate selected in Serial Monitor, or Serial.begin() missing from code                 | Set Serial Monitor dropdown to 9600 baud to match Serial.begin(9600) in the sketch.                                                  |
| LED stays on permanently regardless of distance                             | LED wired directly to 5V instead of the Arduino pin, or code logic reversed                     | Confirm LED anode connects to pin 7 (through resistor), not directly to 5V or GND.                                                   |
| Code uploads but Arduino resets repeatedly or acts erratically              | HC-SR04 drawing too much current, or loose USB connection                                       | Ensure sensor VCC connects to 5V (not 3.3V), and USB cable is fully seated.                                                          |

## Discussion Questions

- What two things does the HC-SR04 sensor actually have instead of "eyes," and what does each one do?
- Why do we divide the total travel time by 2 when calculating distance?
- Can you think of a real-world device that uses the same idea as this sensor (hint: bats, ships, parking sensors)?
- What happens to the LED if you point the sensor at a soft object like a jersey compared to a hard object like a book? Why might that be?

- If you wanted the warning to happen further away (say, 50cm instead of 10cm), which single line of code would you change?

## Extension Challenges

---

- **Easy:** Change the warning distance from 10cm to 30cm and confirm the LED now triggers earlier. Test with a ruler to measure exactly where it turns on.
- **Medium:** Add a second LED (e.g. green) that stays on when the object is far away (safe), so you now have a green "safe" light and a red "warning" light working together based on distance.
- **Hard:** Add a buzzer that beeps faster and faster as an object gets closer, using the distance value to control the delay between beeps (a real parking-sensor effect). Use the distanceCm variable to calculate the beep delay.

## Basic alarm system (*Intermediate* · ~90 min)

### Materials

---

- 1 x Arduino Uno R3 (Nerokas.co.ke)
- 1 x PIR Motion Sensor Module (HC-SR501) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x Active Buzzer Module (5V) (Nerokas.co.ke)
- 1 x Red LED (5mm) (Nerokas.co.ke)
- 1 x Green LED (5mm) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 2 x 220 ohm Resistors (for LEDs) (Nerokas.co.ke)
- 1 x Breadboard (830 tie points) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x Push Button (for arming/disarming the alarm) (Nerokas.co.ke)
- 1 x 10k ohm Resistor (for push button pull-down) (Nerokas.co.ke)
- Male-to-Male Jumper Wires (pack of at least 15) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x USB Cable (Type A to Type B) for Arduino Uno (Nerokas.co.ke)
- 1 x 9V Battery with Arduino Barrel Connector (optional, for portable demo) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))

## How It Works

---

Imagine you have a security guard whose only job is to watch a room and shout if someone walks in who shouldn't be there. In this project, our "guard" is a small sensor called a PIR (Passive Infrared) sensor. This sensor can detect the invisible heat that all living things give off. When a warm body (like a person) moves in front of it, the sensor notices the change in heat and sends a signal to the Arduino saying "Something moved!"

The Arduino is like the guard's brain. It is constantly checking the PIR sensor's signal, over and over, many times per second. This is called **polling** — repeatedly asking "did anything change yet?" When the Arduino sees the "motion detected" signal, it makes a decision in the code (this is called a **conditional** — an "if this happens, then do that" instruction) and reacts by turning on a red LED and sounding a buzzer, just like a real alarm.

We also add a push button that acts as a key switch — pressing it once "arms" the alarm (turns it on and ready), and pressing it again "disarms" it (turns it off), just like you would with a real house alarm keypad. A green LED shows when the system is safely armed and watching.

This project combines three important ideas: **sensing** (the PIR detecting motion), **decision-making** (the Arduino code deciding what to do with that information), and **output** (the buzzer and LEDs reacting). This is exactly how real-world security systems, streetlights, and even automatic doors work.

## Step-by-Step Instructions

---

1. Lay the breadboard flat on the table and place the Arduino next to it, not touching.
2. Insert the PIR sensor's three pins into the breadboard (or connect directly with jumper wires if it has a pin header cable). Identify the VCC, OUT, and GND pins — they are usually labelled on the small circuit board.
3. Connect the PIR sensor's VCC pin to the Arduino's 5V pin using a jumper wire.
4. Connect the PIR sensor's GND pin to the Arduino's GND pin using a jumper wire.
5. Connect the PIR sensor's OUT pin to Arduino digital pin 2 using a jumper wire.

6. Place the red LED on the breadboard. Connect its longer leg (anode) through a 220 ohm resistor to Arduino digital pin 8. Connect its shorter leg (cathode) to the breadboard's GND rail, then to Arduino GND.
7. Place the green LED on the breadboard the same way, but connect its resistor-anode side to Arduino digital pin 9. Connect its cathode to GND.
8. Connect the buzzer's positive pin to Arduino digital pin 7, and its negative pin to GND.
9. Set up the push button: connect one leg to Arduino digital pin 4, and the same leg through a 10k ohm resistor to GND (this is the pull-down resistor, keeping the reading steady when the button is not pressed). Connect the opposite leg of the button to 5V.
10. Double-check all GND connections are shared on a common rail — a loose ground wire is the most common wiring mistake.
11. Connect the Arduino to the computer with the USB cable and open the Arduino IDE.
12. Type or paste the code from the Full Code section below, then upload it to the Arduino.
13. Open the Serial Monitor (set to 9600 baud) to watch status messages as you test the system.
14. Press the push button once. Observe the green LED turn on — this means the alarm is now "armed."
15. Wait about 15-20 seconds without moving — PIR sensors need a short warm-up/calibration time when first powered.
16. Wave your hand or walk in front of the PIR sensor. Observe the red LED turn on and the buzzer sound — this is the alarm triggering.
17. Press the push button again to disarm the system. Observe the green LED turn off, and confirm the buzzer stops responding to motion.
18. Let students take turns arming, triggering, and disarming the system while explaining out loud what they think is happening at each step.

## Full Code

```
// ----- Basic Alarm System -----  
// Motion sensor triggers buzzer & LED alarm  
// Push button arms/disarms the system
```

```

const int pirPin = 2;      // PIR sensor OUTPUT pin
const int buzzerPin = 7;   // Buzzer signal pin
const int redLedPin = 8;   // Red LED = alarm triggered
const int greenLedPin = 9; // Green LED = system armed
const int buttonPin = 4;   // Push button pin

bool systemArmed = false;   // Tracks whether alarm is currently armed
bool lastButtonState = LOW; // Stores previous button reading to detect a new press
bool alarmTriggered = false; // Tracks whether alarm is currently sounding

void setup() {
  pinMode(pirPin, INPUT);      // PIR sends HIGH/LOW signal, so it's an input
  pinMode(buzzerPin, OUTPUT);   // We control the buzzer, so it's an output
  pinMode(redLedPin, OUTPUT);   // We control the red LED
  pinMode(greenLedPin, OUTPUT); // We control the green LED
  pinMode(buttonPin, INPUT);    // Button reading comes into the Arduino

  Serial.begin(9600); // Start Serial communication so we can see debug messages
  Serial.println("System ready. Press button to arm the alarm.");

  delay(15000); // Give the PIR sensor time to calibrate to the room's normal heat levels
  Serial.println("PIR sensor calibrated and ready.");
}

void loop() {
  bool currentButtonState = digitalRead(buttonPin); // Read the button right now

  // Detect a NEW press (button just went from LOW to HIGH)
  if (currentButtonState == HIGH && lastButtonState == LOW) {
    systemArmed = !systemArmed; // Flip the armed status every time button is newly pressed
    Serial.println(systemArmed ? "System ARMED" : "System DISARMED");
    delay(50); // Small delay to avoid reading the same press twice (debounce)
  }

  lastButtonState = currentButtonState; // Save this reading for next loop comparison

  digitalWrite(greenLedPin, systemArmed ? HIGH : LOW); // Green LED shows armed status

  if (systemArmed) {
    int motionDetected = digitalRead(pirPin); // Check what the PIR sensor sees

    if (motionDetected == HIGH) {
      alarmTriggered = true; // Motion found while armed = trigger the alarm
    }

    if (alarmTriggered) {
      digitalWrite(redLedPin, HIGH); // Turn on red LED
      digitalWrite(buzzerPin, HIGH); // Turn on buzzer sound
      Serial.println("ALARM! Motion detected!");
    }
  } else {
    // If system is disarmed, make sure alarm outputs are off
    alarmTriggered = false;
    digitalWrite(redLedPin, LOW);
    digitalWrite(buzzerPin, LOW);
  }

  delay(100); // Short pause to keep loop from running too fast and flooding Serial Monitor
}

```

## Common Mistakes & Fixes

| SYMPTOM                                                                  | LIKELY CAUSE                                                                                               | FIX                                                                                                                               |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Alarm triggers constantly, even with no movement                         | PIR sensor was not given enough calibration/warm-up time, or its sensitivity potentiometer is set too high | Wait the full 15-20 seconds after power-up before testing; turn the sensitivity dial on the PIR module counter-clockwise slightly |
| Nothing happens at all — no LEDs, no buzzer, no Serial messages          | Loose USB connection or code did not upload successfully                                                   | Check the Arduino IDE for an upload error message; reconnect USB cable; confirm correct board and port are selected               |
| Button press doesn't arm/disarm the system, or arms and disarms randomly | Missing pull-down resistor causing "floating" voltage readings on the button pin                           | Confirm the 10k ohm resistor is correctly wired between the button pin and GND                                                    |
| LEDs stay dim or don't light up fully                                    | LED wired backwards (reversed polarity) or missing/wrong resistor value                                    | Flip the LED around so the longer leg (anode) connects to the resistor/pin side; confirm 220 ohm resistor is used                 |
| Buzzer stays silent even though red LED lights up                        | Buzzer wired to wrong pin, or using a passive buzzer that needs a tone() signal instead of HIGH/LOW        | Confirm buzzer type is "active" (usually has a sticker/dome top); check wiring matches pin 7 in the code                          |
| Alarm won't turn off even after disarming with the button                | alarmTriggered variable not resetting because of a logic error in code                                     | Confirm the else block correctly sets alarmTriggered = false when systemArmed is false                                            |

## Discussion Questions

- What does the PIR sensor actually detect — is it seeing light, sound, or something else?
- Why did we need to wait 15-20 seconds before testing the alarm? What might happen if we skipped that wait?
- What is the purpose of the pull-down resistor on the push button? What do you think would happen if we removed it?
- Can you think of a real place in your home, school, or neighbourhood where a system like this could actually be useful?
- If you wanted the alarm to also send a message to someone's phone, what extra parts or ideas do you think you'd need?

## Extension Challenges

---

- **Easy:** Add a second buzzer tone pattern (using `tone()` and `noTone()`) so the alarm makes a beeping sound instead of a constant tone.
- **Medium:** Add a delay so that once armed, the system waits 10 seconds before actually starting to watch for motion (giving the "owner" time to leave the room, like a real house alarm).
- **Hard:** Add an LCD screen or use the Serial Monitor to display a live countdown timer, an "entry delay" before the alarm sounds, and a running log of how many times the alarm has been triggered since the Arduino was powered on.

### 10-12 yrs — 23 children in 3 group(s)

**Projects for this band:** Ultrasonic distance sensor, Basic alarm system, Traffic light simulation. Each group's lead facilitator preps the kit, keeps the group to the master timetable, and watches for the common mistakes noted in each project.

### **Ultrasonic distance sensor** (*Intermediate* · ~75 min)

## Materials

---

- 1 x Arduino Uno (or Uno-compatible board) (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x HC-SR04 Ultrasonic Distance Sensor module (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x Breadboard (half-size or full-size) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x LED (5mm, any colour, red recommended for a "warning" light) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x 220Ω resistor (for the LED) (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x Buzzer (optional, for an audible warning extension) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 8 x Male-to-male jumper wires (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x USB cable (A to B, for Arduino Uno) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x Laptop or computer with Arduino IDE installed (school-provided)
- Optional: ruler or tape measure for calibration activity (school-provided)



## How It Works

---

The HC-SR04 sensor works a lot like a bat finding its way in the dark, or a ship's sonar finding rocks underwater. It has two round "eyes" on the front — but they aren't eyes at all, they are a tiny speaker and a tiny microphone.

The speaker (called the **Trigger**) sends out a very short, very high-pitched sound — so high that humans cannot hear it. This sound travels through the air until it hits an object (a hand, a wall, a book) and bounces back, just like an echo in a canyon. The microphone (called the **Echo**) listens for that bounced-back sound.

The Arduino keeps track of exactly how long it took for the sound to leave and come back — this is measured in tiny fractions of a second called microseconds. Since we know how fast sound travels through air (about 343 metres per second), we can do some simple maths to turn "time taken" into "distance to the object." The Arduino does this maths for us every time it takes a reading.

Once the Arduino knows the distance, it makes a decision: if the object is closer than a distance we choose (say, 10cm), it turns on an LED to warn that something is too close — just like a reversing sensor in a car. This teaches an important programming idea: using **sensor data to make decisions** with an `if` statement.

## Step-by-Step Instructions

---

1. Introduce the HC-SR04 module to students. Point out its four pins: VCC, Trig, Echo, and GND. Explain that Trig "shouts" and Echo "listens."
2. Place the HC-SR04 sensor on the breadboard (or hold it steady with wires — it does not need the breadboard if pins are used directly, but breadboard placement keeps wiring tidy).
3. Wire the sensor: VCC to Arduino 5V, GND to Arduino GND, Trig to Arduino digital pin 9, Echo to Arduino digital pin 10.
4. Insert the LED into the breadboard. Connect the LED's longer leg (anode) through the 220Ω resistor to Arduino digital pin 7. Connect the shorter leg (cathode) to a breadboard ground rail, then to Arduino GND.
5. Double-check all wiring before connecting power — a wrong connection on the HC-SR04 will not usually damage it, but good habits matter.

6. Connect the Arduino to the computer via USB. Open the Arduino IDE and paste in the sketch from the Full Code section below.
7. Select the correct board (Arduino Uno) and correct COM port under the Tools menu.
8. Upload the code. Open the Serial Monitor (set to 9600 baud) to see live distance readings printed in centimetres.
9. Ask students to slowly move a hand towards the sensor while watching the Serial Monitor numbers decrease.
10. Have students note the distance (in cm) at which the LED turns on. Ask them to guess this number before checking.
11. Challenge students to change the threshold value in the code (e.g. from 10cm to 20cm) and re-upload to see the warning distance change.
12. Wrap up by asking each group to explain, in their own words, how the sensor "sees" without eyes.

## Full Code

```
// Ultrasonic Distance Sensor with LED Proximity Warning
// HC-SR04 measures distance; LED lights up when an object is too close

const int trigPin = 9;      // Pin connected to HC-SR04 Trig (sends the sound pulse)
const int echoPin = 10;     // Pin connected to HC-SR04 Echo (receives the bounced sound)
const int ledPin = 7;       // Pin connected to the warning LED

const int warningDistance = 10; // Distance in cm below which the LED turns on

void setup() {
  pinMode(trigPin, OUTPUT); // Trig sends a signal out, so it is an OUTPUT
  pinMode(echoPin, INPUT);  // Echo receives a signal, so it is an INPUT
  pinMode(ledPin, OUTPUT);  // LED is controlled by the Arduino, so it is an OUTPUT
  Serial.begin(9600);       // Start Serial communication so we can see readings on the computer
}

void loop() {
  // Step 1: Send a short pulse out of the Trig pin to trigger a measurement
  digitalWrite(trigPin, LOW); // Make sure Trig starts LOW (off)
  delayMicroseconds(2);       // Tiny pause to ensure a clean signal
  digitalWrite(trigPin, HIGH); // Send the ultrasonic "shout"
  delayMicroseconds(10);      // Pulse must be at least 10 microseconds long
  digitalWrite(trigPin, LOW);  // Turn the trigger off again

  // Step 2: Measure how long the Echo pin stays HIGH
  // pulseIn() waits for the pin to go HIGH, then times how long it stays HIGH (in microseconds)
  long duration = pulseIn(echoPin, HIGH);

  // Step 3: Convert the time into a distance
  // Sound travels about 0.0343 cm per microsecond
  // We divide by 2 because the sound travels to the object AND back
```

```

float distanceCm = (duration * 0.0343) / 2;

// Step 4: Print the distance so we can see it on the Serial Monitor
Serial.print("Distance: ");
Serial.print(distanceCm);
Serial.println(" cm");

// Step 5: Decide whether to warn with the LED
if (distanceCm > 0 && distanceCm < warningDistance) {
  // distanceCm > 0 filters out invalid/zero readings from sensor errors
  digitalWrite(ledPin, HIGH); // Object too close - turn LED on
} else {
  digitalWrite(ledPin, LOW); // Object far away - turn LED off
}

delay(200); // Small pause between readings so numbers don't scroll too fast to read
}

```

## Common Mistakes & Fixes

| SYMPTOM                                                                     | LIKELY CAUSE                                                                                    | FIX                                                                                                                                  |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Serial Monitor shows "0.00 cm" constantly, or numbers that never change     | Trig and Echo pins swapped, or wires loose in breadboard                                        | Double-check wiring: Trig to pin 9, Echo to pin 10. Push wires firmly into breadboard holes.                                         |
| LED never lights up even when an object is very close                       | LED inserted backwards (wrong polarity), or warningDistance value not matched to test distance  | Flip the LED so the longer leg (anode) faces the resistor/pin side. Try moving object closer than the warningDistance value in code. |
| Readings jump wildly or show huge/impossible numbers (e.g. 400+ cm indoors) | Sensor pointed at a soft, angled, or sound-absorbing surface (like fabric), or nothing in range | Point sensor at a flat, hard surface directly in front of it, within 2cm-400cm range.                                                |
| Nothing appears in Serial Monitor at all                                    | Wrong baud rate selected in Serial Monitor, or Serial.begin() missing from code                 | Set Serial Monitor dropdown to 9600 baud to match Serial.begin(9600) in the sketch.                                                  |
| LED stays on permanently regardless of distance                             | LED wired directly to 5V instead of the Arduino pin, or code logic reversed                     | Confirm LED anode connects to pin 7 (through resistor), not directly to 5V or GND.                                                   |
| Code uploads but Arduino resets repeatedly or acts erratically              | HC-SR04 drawing too much current, or loose USB connection                                       | Ensure sensor VCC connects to 5V (not 3.3V), and USB cable is fully seated.                                                          |

## Discussion Questions

---

- What two things does the HC-SR04 sensor actually have instead of "eyes," and what does each one do?
- Why do we divide the total travel time by 2 when calculating distance?
- Can you think of a real-world device that uses the same idea as this sensor (hint: bats, ships, parking sensors)?
- What happens to the LED if you point the sensor at a soft object like a jersey compared to a hard object like a book? Why might that be?
- If you wanted the warning to happen further away (say, 50cm instead of 10cm), which single line of code would you change?

## Extension Challenges

---

- **Easy:** Change the warning distance from 10cm to 30cm and confirm the LED now triggers earlier. Test with a ruler to measure exactly where it turns on.
- **Medium:** Add a second LED (e.g. green) that stays on when the object is far away (safe), so you now have a green "safe" light and a red "warning" light working together based on distance.
- **Hard:** Add a buzzer that beeps faster and faster as an object gets closer, using the distance value to control the delay between beeps (a real parking-sensor effect). Use the distanceCm variable to calculate the beep delay.

## Basic alarm system (*Intermediate · ~90 min*)

### Materials

---

- 1 x Arduino Uno R3 (Nerokas.co.ke)
- 1 x PIR Motion Sensor Module (HC-SR501) (Pixel Electronics - pixelelectronics.co.ke)
- 1 x Active Buzzer Module (5V) (Nerokas.co.ke)
- 1 x Red LED (5mm) (Nerokas.co.ke)
- 1 x Green LED (5mm) (Pixel Electronics - pixelelectronics.co.ke)
- 2 x 220 ohm Resistors (for LEDs) (Nerokas.co.ke)
- 1 x Breadboard (830 tie points) (Pixel Electronics - pixelelectronics.co.ke)
- 1 x Push Button (for arming/disarming the alarm) (Nerokas.co.ke)

- 1 x 10k ohm Resistor (for push button pull-down) (Nerokas.co.ke)
- Male-to-Male Jumper Wires (pack of at least 15) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x USB Cable (Type A to Type B) for Arduino Uno (Nerokas.co.ke)
- 1 x 9V Battery with Arduino Barrel Connector (optional, for portable demo) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))

## How It Works

---

Imagine you have a security guard whose only job is to watch a room and shout if someone walks in who shouldn't be there. In this project, our "guard" is a small sensor called a PIR (Passive Infrared) sensor. This sensor can detect the invisible heat that all living things give off. When a warm body (like a person) moves in front of it, the sensor notices the change in heat and sends a signal to the Arduino saying "Something moved!"

The Arduino is like the guard's brain. It is constantly checking the PIR sensor's signal, over and over, many times per second. This is called **polling** — repeatedly asking "did anything change yet?" When the Arduino sees the "motion detected" signal, it makes a decision in the code (this is called a **conditional** — an "if this happens, then do that" instruction) and reacts by turning on a red LED and sounding a buzzer, just like a real alarm.

We also add a push button that acts as a key switch — pressing it once "arms" the alarm (turns it on and ready), and pressing it again "disarms" it (turns it off), just like you would with a real house alarm keypad. A green LED shows when the system is safely armed and watching.

This project combines three important ideas: **sensing** (the PIR detecting motion), **decision-making** (the Arduino code deciding what to do with that information), and **output** (the buzzer and LEDs reacting). This is exactly how real-world security systems, streetlights, and even automatic doors work.

## Step-by-Step Instructions

---

1. Lay the breadboard flat on the table and place the Arduino next to it, not touching.

2. Insert the PIR sensor's three pins into the breadboard (or connect directly with jumper wires if it has a pin header cable). Identify the VCC, OUT, and GND pins — they are usually labelled on the small circuit board.
3. Connect the PIR sensor's VCC pin to the Arduino's 5V pin using a jumper wire.
4. Connect the PIR sensor's GND pin to the Arduino's GND pin using a jumper wire.
5. Connect the PIR sensor's OUT pin to Arduino digital pin 2 using a jumper wire.
6. Place the red LED on the breadboard. Connect its longer leg (anode) through a 220 ohm resistor to Arduino digital pin 8. Connect its shorter leg (cathode) to the breadboard's GND rail, then to Arduino GND.
7. Place the green LED on the breadboard the same way, but connect its resistor-anode side to Arduino digital pin 9. Connect its cathode to GND.
8. Connect the buzzer's positive pin to Arduino digital pin 7, and its negative pin to GND.
9. Set up the push button: connect one leg to Arduino digital pin 4, and the same leg through a 10k ohm resistor to GND (this is the pull-down resistor, keeping the reading steady when the button is not pressed). Connect the opposite leg of the button to 5V.
10. Double-check all GND connections are shared on a common rail — a loose ground wire is the most common wiring mistake.
11. Connect the Arduino to the computer with the USB cable and open the Arduino IDE.
12. Type or paste the code from the Full Code section below, then upload it to the Arduino.
13. Open the Serial Monitor (set to 9600 baud) to watch status messages as you test the system.
14. Press the push button once. Observe the green LED turn on — this means the alarm is now "armed."
15. Wait about 15-20 seconds without moving — PIR sensors need a short warm-up/calibration time when first powered.
16. Wave your hand or walk in front of the PIR sensor. Observe the red LED turn on and the buzzer sound — this is the alarm triggering.
17. Press the push button again to disarm the system. Observe the green LED turn off, and confirm the buzzer stops responding to motion.

18. Let students take turns arming, triggering, and disarming the system while explaining out loud what they think is happening at each step.

## Full Code

```
// ----- Basic Alarm System -----
// Motion sensor triggers buzzer & LED alarm
// Push button arms/disarms the system

const int pirPin = 2;      // PIR sensor OUTPUT pin
const int buzzerPin = 7;   // Buzzer signal pin
const int redLedPin = 8;   // Red LED = alarm triggered
const int greenLedPin = 9; // Green LED = system armed
const int buttonPin = 4;   // Push button pin

bool systemArmed = false;   // Tracks whether alarm is currently armed
bool lastButtonState = LOW; // Stores previous button reading to detect a new press
bool alarmTriggered = false; // Tracks whether alarm is currently sounding

void setup(){
  pinMode(pirPin, INPUT);      // PIR sends HIGH/LOW signal, so it's an input
  pinMode(buzzerPin, OUTPUT);  // We control the buzzer, so it's an output
  pinMode(redLedPin, OUTPUT);  // We control the red LED
  pinMode(greenLedPin, OUTPUT); // We control the green LED
  pinMode(buttonPin, INPUT);   // Button reading comes into the Arduino

  Serial.begin(9600); // Start Serial communication so we can see debug messages
  Serial.println("System ready. Press button to arm the alarm.");

  delay(15000); // Give the PIR sensor time to calibrate to the room's normal heat levels
  Serial.println("PIR sensor calibrated and ready.");
}

void loop() {
  bool currentButtonState = digitalRead(buttonPin); // Read the button right now

  // Detect a NEW press (button just went from LOW to HIGH)
  if (currentButtonState == HIGH && lastButtonState == LOW) {
    systemArmed = !systemArmed; // Flip the armed status every time button is newly pressed
    Serial.println(systemArmed ? "System ARMED" : "System DISARMED");
    delay(50); // Small delay to avoid reading the same press twice (debounce)
  }

  lastButtonState = currentButtonState; // Save this reading for next loop comparison

  digitalWrite(greenLedPin, systemArmed ? HIGH : LOW); // Green LED shows armed status

  if (systemArmed) {
    int motionDetected = digitalRead(pirPin); // Check what the PIR sensor sees

    if (motionDetected == HIGH) {
      alarmTriggered = true; // Motion found while armed = trigger the alarm
    }

    if (alarmTriggered) {
      digitalWrite(redLedPin, HIGH); // Turn on red LED
    }
  }
}
```

```

    digitalWrite(buzzerPin, HIGH); // Turn on buzzer sound
    Serial.println("ALARM! Motion detected!");
  }
} else {
  // If system is disarmed, make sure alarm outputs are off
  alarmTriggered = false;
  digitalWrite(redLedPin, LOW);
  digitalWrite(buzzerPin, LOW);
}

delay(100); // Short pause to keep loop from running too fast and flooding Serial Monitor
}

```

## Common Mistakes & Fixes

| SYMPTOM                                                                  | LIKELY CAUSE                                                                                               | FIX                                                                                                                               |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Alarm triggers constantly, even with no movement                         | PIR sensor was not given enough calibration/warm-up time, or its sensitivity potentiometer is set too high | Wait the full 15-20 seconds after power-up before testing; turn the sensitivity dial on the PIR module counter-clockwise slightly |
| Nothing happens at all — no LEDs, no buzzer, no Serial messages          | Loose USB connection or code did not upload successfully                                                   | Check the Arduino IDE for an upload error message; reconnect USB cable; confirm correct board and port are selected               |
| Button press doesn't arm/disarm the system, or arms and disarms randomly | Missing pull-down resistor causing "floating" voltage readings on the button pin                           | Confirm the 10k ohm resistor is correctly wired between the button pin and GND                                                    |
| LEDs stay dim or don't light up fully                                    | LED wired backwards (reversed polarity) or missing/wrong resistor value                                    | Flip the LED around so the longer leg (anode) connects to the resistor/pin side; confirm 220 ohm resistor is used                 |
| Buzzer stays silent even though red LED lights up                        | Buzzer wired to wrong pin, or using a passive buzzer that needs a tone() signal instead of HIGH/LOW        | Confirm buzzer type is "active" (usually has a sticker/dome top); check wiring matches pin 7 in the code                          |
| Alarm won't turn off even after disarming with the button                | alarmTriggered variable not resetting because of a logic error in code                                     | Confirm the else block correctly sets alarmTriggered = false when systemArmed is false                                            |

## Discussion Questions

- What does the PIR sensor actually detect — is it seeing light, sound, or something else?



- Why did we need to wait 15-20 seconds before testing the alarm? What might happen if we skipped that wait?
- What is the purpose of the pull-down resistor on the push button? What do you think would happen if we removed it?
- Can you think of a real place in your home, school, or neighbourhood where a system like this could actually be useful?
- If you wanted the alarm to also send a message to someone's phone, what extra parts or ideas do you think you'd need?

## Extension Challenges

---

- **Easy:** Add a second buzzer tone pattern (using `tone()` and `noTone()`) so the alarm makes a beeping sound instead of a constant tone.
- **Medium:** Add a delay so that once armed, the system waits 10 seconds before actually starting to watch for motion (giving the "owner" time to leave the room, like a real house alarm).
- **Hard:** Add an LCD screen or use the Serial Monitor to display a live countdown timer, an "entry delay" before the alarm sounds, and a running log of how many times the alarm has been triggered since the Arduino was powered on.

## Traffic light simulation (*Beginner · ~60 min*)

### Materials

---

- 1 x Arduino Uno board (Nerokas.co.ke)
- 1 x USB cable (Type A to Type B) for programming (Nerokas.co.ke)
- 1 x Breadboard, half-size or full-size (Pixel Electronics.co.ke)
- 1 x Red LED, 5mm (Nerokas.co.ke)
- 1 x Yellow (or amber) LED, 5mm (Nerokas.co.ke)
- 1 x Green LED, 5mm (Nerokas.co.ke)
- 3 x 220 ohm resistors (colour bands: red-red-brown) (Pixel Electronics.co.ke)
- 4 x Male-to-male jumper wires (Nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (not purchased locally)

## How It Works

---

Real traffic lights follow a fixed pattern: red means stop, green means go, and yellow (or amber) warns drivers that the light is about to change. This project recreates that pattern using three LEDs (Light Emitting Diodes) — one red, one yellow, one green — controlled by an Arduino.

An Arduino is a small computer that can turn electricity on and off very precisely, using a program (called a "sketch") that we write and upload to it. Each LED is connected to a different pin on the Arduino. In our code, we tell the Arduino things like "turn on the red LED, wait 5 seconds, turn it off, turn on the green LED..." and so on, in a loop that repeats forever.

LEDs only allow electricity to flow in one direction and they need just the right amount of current — too much and they burn out instantly. That is why each LED is paired with a 220 ohm resistor, which acts like a narrow gate that limits how much electricity can flow through, protecting the LED from damage.

The core programming idea here is **sequencing with timing**: the Arduino executes instructions one after another, and the `delay()` function tells it to pause for a set number of milliseconds before moving to the next instruction. By arranging `digitalWrite()` (turn a pin HIGH or LOW) and `delay()` commands in the right order inside a loop, we recreate the real-world timing of a traffic light.

## Step-by-Step Instructions

---

1. Place the breadboard in front of you with the long power rails running along the top and bottom edges.
2. Insert the red LED into the breadboard. Note that LEDs have one longer leg (the anode, positive) and one shorter leg (the cathode, negative). Place it so its two legs are in different rows of the breadboard.
3. Repeat for the yellow and green LEDs, spacing them out so each has its own set of rows and does not touch its neighbours.
4. For each LED, connect a 220 ohm resistor from the LED's shorter leg (cathode) row to the breadboard's ground (-) rail.
5. Run one jumper wire from the ground (-) rail on the breadboard to the GND pin on the Arduino.

6. Run a jumper wire from the red LED's longer leg (anode) row to digital pin 13 on the Arduino.
7. Run a jumper wire from the yellow LED's longer leg (anode) row to digital pin 12 on the Arduino.
8. Run a jumper wire from the green LED's longer leg (anode) row to digital pin 11 on the Arduino.
9. Double-check that no bare wires are touching each other on the breadboard, and that each LED has its own resistor in its path.
10. Connect the Arduino to the computer using the USB cable.
11. Open the Arduino IDE, select the correct board (Arduino Uno) and port from the Tools menu.
12. Copy the sketch from the "Full Code" section into a new sketch window.
13. Click the Upload button (right-facing arrow icon) and wait for "Done uploading" to appear.
14. Observe: the red LED should light for several seconds, then turn off as green lights up, then green turns off as yellow blinks briefly, then it returns to red — repeating in a continuous loop.
15. If an LED does not light, check that it is inserted the correct way round (long leg towards the Arduino pin side) before assuming the code is wrong.
16. Once working, ask students to time the sequence with a stopwatch and compare it to real traffic lights they have seen.

## Full Code

```
// Traffic Light Simulation
// Three LEDs (red, yellow, green) mimic a real traffic light sequence

// Assign each LED colour to an Arduino digital pin
const int redPin = 13;    // Red LED connected to pin 13
const int yellowPin = 12; // Yellow LED connected to pin 12
const int greenPin = 11;  // Green LED connected to pin 11

// Timing settings (in milliseconds) - easy to change later
const int redDuration = 4000;    // Red light stays on for 4 seconds
const int greenDuration = 4000;  // Green light stays on for 4 seconds
const int yellowDuration = 1000; // Yellow light stays on for 1 second

void setup() {
  // Set all three LED pins as OUTPUT so the Arduino can send power to them
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
}
```

```

pinMode(greenPin, OUTPUT);
}

void loop() {
  // --- RED PHASE: Stop ---
  digitalWrite(redPin, HIGH);    // Turn red LED on
  digitalWrite(yellowPin, LOW);  // Make sure yellow is off
  digitalWrite(greenPin, LOW);   // Make sure green is off
  delay(redDuration);            // Wait while red stays lit

  // --- GREEN PHASE: Go ---
  digitalWrite(redPin, LOW);     // Turn red off
  digitalWrite(greenPin, HIGH);  // Turn green on
  delay(greenDuration);          // Wait while green stays lit

  // --- YELLOW PHASE: Prepare to stop ---
  digitalWrite(greenPin, LOW);   // Turn green off
  digitalWrite(yellowPin, HIGH); // Turn yellow on as a warning
  delay(yellowDuration);         // Wait a short time on yellow
  digitalWrite(yellowPin, LOW);  // Turn yellow off before looping back to red

  // The loop() function repeats automatically, so it goes back to RED PHASE
}

```

## Common Mistakes & Fixes

| SYMPTOM                                                                    | LIKELY CAUSE                                                                 | FIX                                                                                                     |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| An LED does not light up at all                                            | LED inserted backwards (legs reversed)                                       | Remove the LED, flip it around so the longer leg (anode) connects towards the Arduino pin, and reinsert |
| LED glows very faintly or flickers                                         | Loose jumper wire or resistor not fully pushed into breadboard holes         | Press all wires and resistor legs firmly into the breadboard rows                                       |
| All LEDs stay off after uploading                                          | Wrong board or port selected in Arduino IDE, or upload failed silently       | Check Tools > Board and Tools > Port, then re-upload and wait for "Done uploading" message              |
| Two LEDs light up at the same time when they should not                    | digitalWrite(..., LOW) line missing or commented out for the previous colour | Check each phase in the code turns off the LED(s) that should no longer be lit                          |
| Sequence timing looks wrong (too fast or too slow)                         | delay() values not matched to the intended real-world timing                 | Adjust the redDuration, greenDuration and yellowDuration constants at the top of the sketch             |
| LED burns out (goes dark permanently, sometimes with a small pop or smell) | Missing or wrong-value resistor allowed too much current through the LED     | Always use the 220 ohm resistor in series with each LED before power is applied                         |

## Discussion Questions

---

- Why do you think traffic lights use red, yellow, and green specifically, rather than any other three colours?
- What would happen in our code if we forgot to turn off the red LED before turning on the green one?
- Why does the resistor need to be placed in the circuit, and what might happen to the LED without it?
- In the code, what is the purpose of the delay() function, and what do you think would happen if all the delay values were changed to 100 milliseconds?
- Real traffic lights sometimes have a flashing yellow warning mode at night. How could you modify this program to add that behaviour?

## Extension Challenges

---

- **Easy:** Change the timing so red lasts 6 seconds and green lasts 3 seconds, then re-upload and observe the difference.
- **Medium:** Add a push button that, when pressed, immediately switches the light to red (like a pedestrian crossing button), then resumes the normal sequence afterward.
- **Hard:** Build a second traffic light for a perpendicular road using three more LEDs, and write code so that when one light is green, the other is red, simulating a real road intersection.

### 13-15 yrs — 12 children in 2 group(s)

**Projects for this band:** Smart city, Traffic light simulation, Ultrasonic distance sensor. Each group's lead facilitator preps the kit, keeps the group to the master timetable, and watches for the common mistakes noted in each project.

### Smart city (*Advanced* · ~120 min)

## Overview

---

The Smart City build brings three city subsystems onto one Arduino: a **traffic-light intersection**, an **automatic street light** that turns on when it gets dark, and a **smart-parking bay** that senses whether a car is present and shows the status on

an LCD. It is the flagship "Solve" project — students integrate several sensors and outputs into one working model of their town.

## Materials

- Arduino Uno & USB cable — 1 (Nerokas, [nerokas.co.ke](http://nerokas.co.ke))
- Breadboard & jumper wires — 1 set (Nerokas / Pixel Electronics, [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- LEDs: red, yellow, green for traffic + 1 for street light + 2 for parking status — 6 total, with 220Ω resistors
- LDR (light-dependent resistor) + 10kΩ resistor — 1 (voltage divider into A0)
- HC-SR04 ultrasonic sensor — 1 (parking)
- 16x2 LCD (parallel / HD44780) + 10kΩ potentiometer for contrast — 1

*Optional for extensions:* DHT11, MQ135 air-quality, PIR motion, soil-moisture, servo, buzzer, relay, ESP8266/ESP32 for IoT.

## How It Works

- **Traffic lights** cycle green → yellow → red on a timer written as a non-blocking state machine (using `millis()`), so the rest of the city keeps running.
- **Street light** reads the LDR each loop; when the reading drops below the "dark" threshold the LED switches on — just like real dusk-to-dawn lighting.
- **Smart parking** pings the ultrasonic sensor: a car within 10 cm marks the bay *occupied* (red LED), otherwise *free* (green LED), and the LCD shows FREE / FULL.

## Wiring & Pin Map

| PART                         | ARDUINO PIN(S) |
|------------------------------|----------------|
| Traffic red / yellow / green | 13 / 12 / 11   |
| Street-light LED             | 10             |
| Parking free / occupied LED  | 7 / 6          |
| Ultrasonic TRIG / ECHO       | 9 / 8          |
| LDR divider                  | A0             |

| PART                       | ARDUINO PIN(S)     |
|----------------------------|--------------------|
| LCD rs, en, d4, d5, d6, d7 | 2, 3, 4, 5, A1, A2 |

## Step-by-Step Build

1. Build the traffic light first (3 LEDs + resistors) and get the timed sequence running.
2. Add the LDR as a voltage divider into A0; add the street-light LED and test dark/light.
3. Wire the HC-SR04 (TRIG 9, ECHO 8) and the two parking LEDs; test the distance reading on the Serial Monitor.
4. Wire the LCD (contrast pot to the middle pin) and show the parking status on the bottom row.
5. Upload the full sketch below and tune `DARK` and `PARK_CM` for your room.

## Full Code

```
#include

// 16x2 LCD in 4-bit mode: rs, en, d4, d5, d6, d7
LiquidCrystal lcd(2, 3, 4, 5, A1, A2);

// --- Pin map ---
const int RED = 13, YEL = 12, GRN = 11;    // traffic light
const int STREET = 10;                     // smart street light LED
const int TRIG = 9, ECHO = 8;               // parking ultrasonic (HC-SR04)
const int P_FREE = 7, P_OCC = 6;           // parking status LEDs
const int LDR = A0;                         // light sensor (analog)

// --- Tuning ---
const unsigned long GREEN_MS = 5000, YELLOW_MS = 2000, RED_MS = 5000;
const int DARK = 400;                      // LDR reading below this = "night" (tune on the day)
const int PARK_CM = 10;                    // car within 10 cm = bay occupied

int phase = 0;                             // 0 = green, 1 = yellow, 2 = red
unsigned long phaseStart = 0;
unsigned long lastSense = 0;

void setup() {
  int outs[] = {RED, YEL, GRN, STREET, P_FREE, P_OCC};
  for (int i = 0; i < 6; i++) pinMode(outs[i], OUTPUT);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  lcd.begin(16, 2);
  lcd.print("Smart City");
  phaseStart = millis();
}
```

```

// Read distance in cm; returns -1 on timeout (nothing in range).
long readCm() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000); // 30 ms timeout
  return us == 0 ? -1 : us / 58; // ~58 us per cm (round trip)
}

// Non-blocking traffic light so parking + lights keep updating.
void updateTraffic() {
  unsigned long now = millis();
  unsigned long dur = (phase == 0) ? GREEN_MS : (phase == 1) ? YELLOW_MS : RED_MS;
  if (now - phaseStart >= dur) { phase = (phase + 1) % 3; phaseStart = now; }
  digitalWrite(GRN, phase == 0);
  digitalWrite(YEL, phase == 1);
  digitalWrite(RED, phase == 2);
}

void loop() {
  updateTraffic();

  // Smart street light: switch on automatically when it gets dark.
  digitalWrite(STREET, analogRead(LDR) < DARK ? HIGH : LOW);

  // Smart parking: check the bay twice a second.
  if (millis() - lastSense >= 500) {
    lastSense = millis();
    long cm = readCm();
    bool occupied = (cm > 0 && cm <= PARK_CM);
    digitalWrite(P_OCC, occupied);
    digitalWrite(P_FREE, !occupied);
    lcd.setCursor(0, 1);
    lcd.print(occupied ? "Parking: FULL " : "Parking: FREE ");
  }
}

```

## Common Mistakes & Fixes

| SYMPTOM                                 | LIKELY CAUSE                                     | FIX                                                                                                      |
|-----------------------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Everything freezes during the red light | Using <code>delay()</code> for the traffic light | Use the <code>millis()</code> state machine shown here so sensors keep reading                           |
| Street light never turns on/off         | <code>DARK</code> threshold wrong for the room   | Print <code>analogRead(LDR)</code> to Serial and set <code>DARK</code> between the light and dark values |
| Parking always reads FULL or FREE       | ECHO/TRIG swapped, or timeout                    | Check wiring; <code>readCm()</code> returns -1 on timeout — treat as "free"                              |
| LCD shows blocks or nothing             | Contrast pot not set / miswired                  | Turn the 10k pot until text appears; recheck rs/en/d4-d7                                                 |



## Discussion Questions

---

1. Why is a non-blocking (`millis()`) traffic light better than one built with `delay()`?
2. What real problem in your town could each subsystem help with?
3. How could the parking bay count several spaces instead of one?
4. Where would you add a sensor to make the city safer or greener?
5. What are the trade-offs of automating street lights (cost, safety, energy)?

## Extension Challenges

---

- **Easy:** add a second intersection direction; count multiple free parking bays on the LCD.
- **Medium:** add an air-quality monitor (MQ135), a weather station (DHT11), automatic pedestrian lighting (PIR), or a home-security intruder alarm (PIR + buzzer).
- **Hard:** make the traffic lights respond to traffic volume (ultrasonic), add smart irrigation (soil-moisture + relay), or connect to the internet with an ESP8266/ESP32 and plot temperature, traffic and air quality on an Arduino Cloud / Blynk / ThingSpeak dashboard.
- **Capstone:** teams combine subsystems into one miniature city — traffic, smart homes, street lights, parking, environment sensors, an emergency system and an IoT dashboard.

## 12-Week Club Pathway (optional)

---

For clubs and camps that want to run Smart City as a course, this build is the spine of a 12-week pathway:

| WEEK | DELIVERABLE                       |
|------|-----------------------------------|
| 1    | Smart city plan (design thinking) |
| 2    | Blink LED — Arduino basics        |
| 3    | Traffic lights                    |
| 4    | Smart intersection                |
| 5    | Parking system                    |

| WEEK | DELIVERABLE              |
|------|--------------------------|
| 6    | Weather station          |
| 7    | Air-quality monitor      |
| 8    | Automatic street lights  |
| 9    | Smart home               |
| 10   | Smart farm               |
| 11   | IoT dashboard            |
| 12   | Complete smart city demo |

## Traffic light simulation (*Beginner* · ~60 min)

### Materials

- 1 x Arduino Uno board (Nerokas.co.ke)
- 1 x USB cable (Type A to Type B) for programming (Nerokas.co.ke)
- 1 x Breadboard, half-size or full-size (Pixel Electronics.co.ke)
- 1 x Red LED, 5mm (Nerokas.co.ke)
- 1 x Yellow (or amber) LED, 5mm (Nerokas.co.ke)
- 1 x Green LED, 5mm (Nerokas.co.ke)
- 3 x 220 ohm resistors (colour bands: red-red-brown) (Pixel Electronics.co.ke)
- 4 x Male-to-male jumper wires (Nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (not purchased locally)

### How It Works

Real traffic lights follow a fixed pattern: red means stop, green means go, and yellow (or amber) warns drivers that the light is about to change. This project recreates that pattern using three LEDs (Light Emitting Diodes) — one red, one yellow, one green — controlled by an Arduino.

An Arduino is a small computer that can turn electricity on and off very precisely, using a program (called a "sketch") that we write and upload to it. Each LED is connected to a different pin on the Arduino. In our code, we tell the Arduino things

like "turn on the red LED, wait 5 seconds, turn it off, turn on the green LED..." and so on, in a loop that repeats forever.

LEDs only allow electricity to flow in one direction and they need just the right amount of current — too much and they burn out instantly. That is why each LED is paired with a 220 ohm resistor, which acts like a narrow gate that limits how much electricity can flow through, protecting the LED from damage.

The core programming idea here is **sequencing with timing**: the Arduino executes instructions one after another, and the `delay()` function tells it to pause for a set number of milliseconds before moving to the next instruction. By arranging `digitalWrite()` (turn a pin HIGH or LOW) and `delay()` commands in the right order inside a loop, we recreate the real-world timing of a traffic light.

## Step-by-Step Instructions

---

1. Place the breadboard in front of you with the long power rails running along the top and bottom edges.
2. Insert the red LED into the breadboard. Note that LEDs have one longer leg (the anode, positive) and one shorter leg (the cathode, negative). Place it so its two legs are in different rows of the breadboard.
3. Repeat for the yellow and green LEDs, spacing them out so each has its own set of rows and does not touch its neighbours.
4. For each LED, connect a 220 ohm resistor from the LED's shorter leg (cathode) row to the breadboard's ground (-) rail.
5. Run one jumper wire from the ground (-) rail on the breadboard to the GND pin on the Arduino.
6. Run a jumper wire from the red LED's longer leg (anode) row to digital pin 13 on the Arduino.
7. Run a jumper wire from the yellow LED's longer leg (anode) row to digital pin 12 on the Arduino.
8. Run a jumper wire from the green LED's longer leg (anode) row to digital pin 11 on the Arduino.
9. Double-check that no bare wires are touching each other on the breadboard, and that each LED has its own resistor in its path.
10. Connect the Arduino to the computer using the USB cable.

11. Open the Arduino IDE, select the correct board (Arduino Uno) and port from the Tools menu.
12. Copy the sketch from the "Full Code" section into a new sketch window.
13. Click the Upload button (right-facing arrow icon) and wait for "Done uploading" to appear.
14. Observe: the red LED should light for several seconds, then turn off as green lights up, then green turns off as yellow blinks briefly, then it returns to red — repeating in a continuous loop.
15. If an LED does not light, check that it is inserted the correct way round (long leg towards the Arduino pin side) before assuming the code is wrong.
16. Once working, ask students to time the sequence with a stopwatch and compare it to real traffic lights they have seen.

## Full Code

```
// Traffic Light Simulation
// Three LEDs (red, yellow, green) mimic a real traffic light sequence

// Assign each LED colour to an Arduino digital pin
const int redPin = 13;    // Red LED connected to pin 13
const int yellowPin = 12; // Yellow LED connected to pin 12
const int greenPin = 11;  // Green LED connected to pin 11

// Timing settings (in milliseconds) - easy to change later
const int redDuration = 4000;    // Red light stays on for 4 seconds
const int greenDuration = 4000;  // Green light stays on for 4 seconds
const int yellowDuration = 1000; // Yellow light stays on for 1 second

void setup() {
  // Set all three LED pins as OUTPUT so the Arduino can send power to them
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
}

void loop() {
  // --- RED PHASE: Stop ---
  digitalWrite(redPin, HIGH);    // Turn red LED on
  digitalWrite(yellowPin, LOW);  // Make sure yellow is off
  digitalWrite(greenPin, LOW);   // Make sure green is off
  delay(redDuration);            // Wait while red stays lit

  // --- GREEN PHASE: Go ---
  digitalWrite(redPin, LOW);     // Turn red off
  digitalWrite(greenPin, HIGH);  // Turn green on
  delay(greenDuration);          // Wait while green stays lit

  // --- YELLOW PHASE: Prepare to stop ---
}
```

```
digitalWrite(greenPin, LOW);    // Turn green off
digitalWrite(yellowPin, HIGH); // Turn yellow on as a warning
delay(yellowDuration);         // Wait a short time on yellow
digitalWrite(yellowPin, LOW);   // Turn yellow off before looping back to red

// The loop() function repeats automatically, so it goes back to RED PHASE
}
```

## Common Mistakes & Fixes

| SYMPTOM                                                                    | LIKELY CAUSE                                                                 | FIX                                                                                                     |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| An LED does not light up at all                                            | LED inserted backwards (legs reversed)                                       | Remove the LED, flip it around so the longer leg (anode) connects towards the Arduino pin, and reinsert |
| LED glows very faintly or flickers                                         | Loose jumper wire or resistor not fully pushed into breadboard holes         | Press all wires and resistor legs firmly into the breadboard rows                                       |
| All LEDs stay off after uploading                                          | Wrong board or port selected in Arduino IDE, or upload failed silently       | Check Tools > Board and Tools > Port, then re-upload and wait for "Done uploading" message              |
| Two LEDs light up at the same time when they should not                    | digitalWrite(..., LOW) line missing or commented out for the previous colour | Check each phase in the code turns off the LED(s) that should no longer be lit                          |
| Sequence timing looks wrong (too fast or too slow)                         | delay() values not matched to the intended real-world timing                 | Adjust the redDuration, greenDuration and yellowDuration constants at the top of the sketch             |
| LED burns out (goes dark permanently, sometimes with a small pop or smell) | Missing or wrong-value resistor allowed too much current through the LED     | Always use the 220 ohm resistor in series with each LED before power is applied                         |

## Discussion Questions

- Why do you think traffic lights use red, yellow, and green specifically, rather than any other three colours?
- What would happen in our code if we forgot to turn off the red LED before turning on the green one?
- Why does the resistor need to be placed in the circuit, and what might happen to the LED without it?
- In the code, what is the purpose of the delay() function, and what do you think would happen if all the delay values were changed to 100 milliseconds?

- Real traffic lights sometimes have a flashing yellow warning mode at night. How could you modify this program to add that behaviour?

## Extension Challenges

---

- **Easy:** Change the timing so red lasts 6 seconds and green lasts 3 seconds, then re-upload and observe the difference.
- **Medium:** Add a push button that, when pressed, immediately switches the light to red (like a pedestrian crossing button), then resumes the normal sequence afterward.
- **Hard:** Build a second traffic light for a perpendicular road using three more LEDs, and write code so that when one light is green, the other is red, simulating a real road intersection.

## Ultrasonic distance sensor (*Intermediate* · ~75 min)

### Materials

---

- 1 x Arduino Uno (or Uno-compatible board) (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x HC-SR04 Ultrasonic Distance Sensor module (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x Breadboard (half-size or full-size) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x LED (5mm, any colour, red recommended for a "warning" light) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x 220Ω resistor (for the LED) (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x Buzzer (optional, for an audible warning extension) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 8 x Male-to-male jumper wires (Nerokas - [nerokas.co.ke](http://nerokas.co.ke))
- 1 x USB cable (A to B, for Arduino Uno) (Pixel Electronics - [pixelelectronics.co.ke](http://pixelelectronics.co.ke))
- 1 x Laptop or computer with Arduino IDE installed (school-provided)
- Optional: ruler or tape measure for calibration activity (school-provided)

### How It Works

---

The HC-SR04 sensor works a lot like a bat finding its way in the dark, or a ship's sonar finding rocks underwater. It has two round "eyes" on the front — but they aren't eyes at all, they are a tiny speaker and a tiny microphone.

The speaker (called the **Trigger**) sends out a very short, very high-pitched sound — so high that humans cannot hear it. This sound travels through the air until it hits an object (a hand, a wall, a book) and bounces back, just like an echo in a canyon. The microphone (called the **Echo**) listens for that bounced-back sound.

The Arduino keeps track of exactly how long it took for the sound to leave and come back — this is measured in tiny fractions of a second called microseconds. Since we know how fast sound travels through air (about 343 metres per second), we can do some simple maths to turn "time taken" into "distance to the object." The Arduino does this maths for us every time it takes a reading.

Once the Arduino knows the distance, it makes a decision: if the object is closer than a distance we choose (say, 10cm), it turns on an LED to warn that something is too close — just like a reversing sensor in a car. This teaches an important programming idea: using **sensor data to make decisions** with an `if` statement.

## Step-by-Step Instructions

---

1. Introduce the HC-SR04 module to students. Point out its four pins: VCC, Trig, Echo, and GND. Explain that Trig "shouts" and Echo "listens."
2. Place the HC-SR04 sensor on the breadboard (or hold it steady with wires — it does not need the breadboard if pins are used directly, but breadboard placement keeps wiring tidy).
3. Wire the sensor: VCC to Arduino 5V, GND to Arduino GND, Trig to Arduino digital pin 9, Echo to Arduino digital pin 10.
4. Insert the LED into the breadboard. Connect the LED's longer leg (anode) through the 220 $\Omega$  resistor to Arduino digital pin 7. Connect the shorter leg (cathode) to a breadboard ground rail, then to Arduino GND.
5. Double-check all wiring before connecting power — a wrong connection on the HC-SR04 will not usually damage it, but good habits matter.
6. Connect the Arduino to the computer via USB. Open the Arduino IDE and paste in the sketch from the Full Code section below.
7. Select the correct board (Arduino Uno) and correct COM port under the Tools menu.
8. Upload the code. Open the Serial Monitor (set to 9600 baud) to see live distance readings printed in centimetres.

9. Ask students to slowly move a hand towards the sensor while watching the Serial Monitor numbers decrease.
10. Have students note the distance (in cm) at which the LED turns on. Ask them to guess this number before checking.
11. Challenge students to change the threshold value in the code (e.g. from 10cm to 20cm) and re-upload to see the warning distance change.
12. Wrap up by asking each group to explain, in their own words, how the sensor "sees" without eyes.

## Full Code

```
// Ultrasonic Distance Sensor with LED Proximity Warning
// HC-SR04 measures distance; LED lights up when an object is too close

const int trigPin = 9;      // Pin connected to HC-SR04 Trig (sends the sound pulse)
const int echoPin = 10;     // Pin connected to HC-SR04 Echo (receives the bounced sound)
const int ledPin = 7;       // Pin connected to the warning LED

const int warningDistance = 10; // Distance in cm below which the LED turns on

void setup() {
  pinMode(trigPin, OUTPUT); // Trig sends a signal out, so it is an OUTPUT
  pinMode(echoPin, INPUT);  // Echo receives a signal, so it is an INPUT
  pinMode(ledPin, OUTPUT);  // LED is controlled by the Arduino, so it is an OUTPUT
  Serial.begin(9600);       // Start Serial communication so we can see readings on the computer
}

void loop() {
  // Step 1: Send a short pulse out of the Trig pin to trigger a measurement
  digitalWrite(trigPin, LOW); // Make sure Trig starts LOW (off)
  delayMicroseconds(2);       // Tiny pause to ensure a clean signal
  digitalWrite(trigPin, HIGH); // Send the ultrasonic "shout"
  delayMicroseconds(10);       // Pulse must be at least 10 microseconds long
  digitalWrite(trigPin, LOW);  // Turn the trigger off again

  // Step 2: Measure how long the Echo pin stays HIGH
  // pulseIn() waits for the pin to go HIGH, then times how long it stays HIGH (in microseconds)
  long duration = pulseIn(echoPin, HIGH);

  // Step 3: Convert the time into a distance
  // Sound travels about 0.0343 cm per microsecond
  // We divide by 2 because the sound travels to the object AND back
  float distanceCm = (duration * 0.0343) / 2;

  // Step 4: Print the distance so we can see it on the Serial Monitor
  Serial.print("Distance: ");
  Serial.print(distanceCm);
  Serial.println(" cm");

  // Step 5: Decide whether to warn with the LED
  if (distanceCm > 0 && distanceCm < warningDistance) {
    // distanceCm > 0 filters out invalid/zero readings from sensor errors
```



```

    digitalWrite(ledPin, HIGH); // Object too close - turn LED on
  } else {
    digitalWrite(ledPin, LOW); // Object far away - turn LED off
  }

  delay(200); // Small pause between readings so numbers don't scroll too fast to read
}

```

## Common Mistakes & Fixes

| SYMPTOM                                                                     | LIKELY CAUSE                                                                                    | FIX                                                                                                                                  |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Serial Monitor shows "0.00 cm" constantly, or numbers that never change     | Trig and Echo pins swapped, or wires loose in breadboard                                        | Double-check wiring: Trig to pin 9, Echo to pin 10. Push wires firmly into breadboard holes.                                         |
| LED never lights up even when an object is very close                       | LED inserted backwards (wrong polarity), or warningDistance value not matched to test distance  | Flip the LED so the longer leg (anode) faces the resistor/pin side. Try moving object closer than the warningDistance value in code. |
| Readings jump wildly or show huge/impossible numbers (e.g. 400+ cm indoors) | Sensor pointed at a soft, angled, or sound-absorbing surface (like fabric), or nothing in range | Point sensor at a flat, hard surface directly in front of it, within 2cm-400cm range.                                                |
| Nothing appears in Serial Monitor at all                                    | Wrong baud rate selected in Serial Monitor, or Serial.begin() missing from code                 | Set Serial Monitor dropdown to 9600 baud to match Serial.begin(9600) in the sketch.                                                  |
| LED stays on permanently regardless of distance                             | LED wired directly to 5V instead of the Arduino pin, or code logic reversed                     | Confirm LED anode connects to pin 7 (through resistor), not directly to 5V or GND.                                                   |
| Code uploads but Arduino resets repeatedly or acts erratically              | HC-SR04 drawing too much current, or loose USB connection                                       | Ensure sensor VCC connects to 5V (not 3.3V), and USB cable is fully seated.                                                          |

## Discussion Questions

- What two things does the HC-SR04 sensor actually have instead of "eyes," and what does each one do?
- Why do we divide the total travel time by 2 when calculating distance?
- Can you think of a real-world device that uses the same idea as this sensor (hint: bats, ships, parking sensors)?

- What happens to the LED if you point the sensor at a soft object like a jersey compared to a hard object like a book? Why might that be?
- If you wanted the warning to happen further away (say, 50cm instead of 10cm), which single line of code would you change?

## Extension Challenges

---

- **Easy:** Change the warning distance from 10cm to 30cm and confirm the LED now triggers earlier. Test with a ruler to measure exactly where it turns on.
- **Medium:** Add a second LED (e.g. green) that stays on when the object is far away (safe), so you now have a green "safe" light and a red "warning" light working together based on distance.
- **Hard:** Add a buzzer that beeps faster and faster as an object gets closer, using the distance value to control the delay between beeps (a real parking-sensor effect). Use the distanceCm variable to calculate the beep delay.

## Materials — Scaling

---

Each project's materials list above is **per group**. Multiply it by the number of groups in that band to get the totals to prepare. Shared kit (laptops, power strips, extension leads) can rotate between groups rather than be bought per group.

## Safety, Logistics & Contingencies

---

- **Space:** one table per group with room to move; keep walkways clear and power leads taped down.
- **Movement:** groups start and stop together on the master timetable — stagger the break and lunch queues so the space never crowds.
- **First aid & allergies:** collect allergy info at registration; the first-aider holds the kit and the contact list all day.
- **Wet weather:** move outdoor free-play to the hall; keep one indoor back-up game ready.
- **Early finishers:** use each project's extension challenges (listed in the project pages below).
- **Equipment failure:** keep one spare kit per band; a failed build becomes a debugging lesson, not a lost hour.

## Staff Briefing Checklist

---

- Every facilitator knows their group, their band, and their room.
- Kits counted and laid out per group before doors open.
- Registration desk has the sign-in sheet, badges, and allergy list.
- Timekeeper owns the master timetable and calls transitions out loud.
- Floating helpers know which groups they cover during breaks.
- Showcase space and closing certificates ready before the last block.

## Organiser Notes

---

Projects were chosen to fit your notes where possible: *AI AND ROBOTICS PROJECTS*